

Criptografia Aplicada: Java Cryptography Architecture (JCA)

O que é a JCA?

A JCA (Java Cryptography Architecture) é uma framework da linguagem Java que suporta a implementação e utilização de criptografia, baseada em:

- **Providers:** conjuntos de algoritmos disponibilizados (hash, cifra, assinatura, geração de chaves).
- **Classes abstratas:** definem a interface e comportamento esperado dos mecanismos criptográficos.

Cada provider precisa ser **registrado** no sistema para ser usado.

Princípios da JCA

- **Independência:** aplicações não implementam os algoritmos, apenas os utilizam via providers.
 - **Interoperabilidade:** qualquer provider que siga a interface pode ser usado de forma transparente.
 - **Extensibilidade:** é possível criar novos providers ou algoritmos.
-

História da JCA

Dividida originalmente em dois componentes:

- **Java Security API:** `java.security.*` (hash e assinatura)
- **Java Cryptography Extension (JCE):** `javax.crypto.*` (cifra e troca de chaves)

Inicialmente restrita a exportações fora dos EUA.

Situação Atual

- JCE é distribuída com o JDK (desde JDK 9 com algoritmos fortes por omissão).
- Versões antigas do JDK (antes do 6u181, 7u171, 8u161) requerem a **Unlimited Strength Jurisdiction Policy**.

Providers base incluídos:

- Sun, SunRsaSign, SunJCE, etc.
- Apenas para testes ou contextos de baixa exigência.

Mais informações:

<http://docs.oracle.com/javase/8/docs/technotes/guides/security/SunProviders.html>

Documentação Oficial

- [CryptoSpec](#)
 - [java.security](#)
 - [javax.crypto](#)
-

Providers – Seleção

Existem providers adicionais (gratuitos e comerciais), que oferecem:

- Implementações otimizadas e seguras
- Suporte a ASN.1, X.509, OCSP, e-mail cifrado/assinado, timestamp

Exemplos

BouncyCastle

- Open-source, muito completo.
- Classe: `org.bouncycastle.jce.provider.BouncyCastleProvider`
- Documentação:
 - <http://www.bouncycastle.org/specifications.html>

IAIK

- Toolkit comercial da Universidade de Graz (Áustria)
 - Classe: `iaik.security.provider.IAIK`
 - Documentação:
 - http://javadoc.iaik.tugraz.at/iaik_jce/current/index.html
-

Providers – Instalação

Estática

- Editar o ficheiro `java.security`
- Adicionar linha: `security.provider.n=nome.da.classe.Principal`

Vantagem:

- Transparente para a aplicação.

Desvantagem:

- Requer acesso à configuração do sistema.

Dinâmica

- Através do código Java:

Java

```
1 Security.addProvider(new BouncyCastleProvider());
2 Security.insertProviderAt(new BouncyCastleProvider(), 1);
```

Utilização de Providers

Implícita

Java

```
1 MessageDigest md = MessageDigest.getInstance("SHA256");
```

Explícita

Java

```
1 MessageDigest md = MessageDigest.getInstance("SHA256", "BC");
```

Útil em contextos de elevada segurança para garantir controlo total.

Componentes Utilitários da JCA

- `java.security.Security`: gerir providers e propriedades.
 - `java.security.Key`: interface comum a todas as chaves.
 - **SecretKey**: cifras simétricas
 - **PrivateKey** / **PublicKey**: cifras assimétricas
-

Geração de Aleatórios

Classe: `java.security.SecureRandom`

- `getInstance(String)`
 - `setSeed(long)`
 - `nextBytes(byte[])`
-

Geração de Chaves

Simétricas

Classe: `javax.crypto.KeyGenerator`

Java

```
1 KeyGenerator keyGen = KeyGenerator.getInstance("AES");
2 keyGen.init(128, new SecureRandom());
3 SecretKey secretKey = keyGen.generateKey();
```

Assimétricas

Classe: `java.security.KeyPairGenerator`

Java

```
1 KeyPairGenerator keyPairGen = KeyPairGenerator.getInstance("RSA");
2 keyPairGen.initialize(2048, new SecureRandom());
3 KeyPair keyPair = keyPairGen.generateKeyPair();
```

Cifragem / Decifragem

Classe: `javax.crypto.Cipher`

Java

```
1 Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
2 cipher.init(Cipher.ENCRYPT_MODE, chave);
3 byte[] resultado = cipher.doFinal(mensagem);
```

Funções de Hash

Classe: `java.security.MessageDigest`

Java

```
1 MessageDigest md = MessageDigest.getInstance("SHA-256");
2 md.update(dados);
```

```
3 byte[] hash = md.digest();
```

MAC (Message Authentication Code)

Classe: `javax.crypto.Mac`

Java

```
1 Mac mac = Mac.getInstance("HmacSHA256");
2 mac.init(secretKey);
3 mac.update(dados);
4 byte[] resultado = mac.doFinal();
```

Assinaturas Digitais

Classe: `java.security.Signature`

Java

```
1 Signature sig = Signature.getInstance("SHA256withRSA");
2 sig.initSign(privateKey);
3 sig.update(dados);
4 byte[] assinatura = sig.sign();
5
6 // Verificação
7 sig.initVerify(publicKey);
8 sig.update(dados);
9 boolean valido = sig.verify(assinatura);
```